# Report Project Algorithms for Massive Datasets

Alessandro Crippa - 84583A

March 2026

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study. No generative AI tool has been used to write the code or the report content.

## 1 Introduction

The project I decided to develop is Project 3, on Link Analysis. Specifically, I chose to implement the PageRank algorithm on a user graph built from the New York Times Articles and Comments dataset.
I implemented PageRank algorithm in two ways: the first using NumPy and Scipy's matrix operations and the second one using Apache Spark. Then, I compared the two methods in terms of runtime, convergence and final results.
Finally, I implemented Topic-Sensitive PageRank, calculated independently for some sections of articles and compare these specific rankings with the PageRank.

## 2 Dataset

The dataset used is the "New York Times Articles and Comments" (2020), published on Kaggle. This dataset was accessed in March 2026 and, since it is dynamically updated, results may slightly vary depending on the download time.
It consists of sveral CSV files, but in this project only two CSV files are considered: "nyt-articles-2020.csv", containing articles with columns "uniqueID" and "section"; "nyt-comments-2020.csv", that contains all the user comments with columns "userID", "articleID", "isAnonymous" and "status". For this project, only the columns "userID", "articleID" and "section" were used.
Since "nyt-comments-2020.csv" is a large file, it is read in chunks of 100000 rows at a time rather than loading the entire file into memory at once. This approach

keeps memory usage limited regardless of the dataset size and allows the code to be robust even in environments with limited resources.

# 3 Data organization

The graph is constructed based on a co-commenting relationship: two users are connected if and only if they commented on at least one common article. This relationship is considered as an undirected relationship: however, since the PagerRank algorithm is define for directed graphs, each undirected edge is modeled as two reciprocal directed edges.
The graph is represented as a sparse adjacency matrix in CSR format (scipy.sparse.csr_matrix) of size $N \times N$, where N denotes the number of distinct users. As most pairs of users never comment on the same article, a lot of the entries of the matrix are zero and the matrix is highly sparse. Storing a dense $N \times N$ matrix would require $N^2$ entries, which becomes impractial for large values of N. Instead, the sparse matrix stores only the non-zero entries, significantly reducing memory usage. Due to the nature of the co-commenting relationship, the matrix is symmetric.
An entry $A_{ij} > 0$ indicates that users i and j have commented on at least one article in common and the value in $A_{ij}$ represents the number of articles that i and j have co-commented. However, once the graph is built, the matrix is binarized and the final value of $A_{ij}$ is 1 if the two users have commented on at least one article in common, 0 otherwise.
To avoid generating an excessive number of edges for articles with many commenters, which would produce a large number of edges and degrade performance, a cap of "MAX USERS PER ARTICLE = 200" was imposed: this means that for article with more 200 commenters, only a random sample of 200 users is considered.
For the Apache Spark implementation, the matrix is converted to an RDD of (src, dst) edge pairs and reorganized into an adjacency list RDD of the form (node, [neighbors]).

# 4 Pre-processing

Before building the graph, several pre-processing steps were applied to ensure data quality.
The comments file, as previously stated, is read in chunks of 100000 rows at time to avoid loading the entire file into memory. Within each chunk, two filters are applied: comments marked as anonymous ("isAnonymous == False") are removed since they can not be attributed to a specific users and comments with a "status" other than "approved" are excluded as they represent rejected or not yet published interactions. After concatenating all filtered chunks, duplicate ("userID, articleID") pairs are dropped: since what matters is whether two users commented on the same article, keeping duplicates would increase the

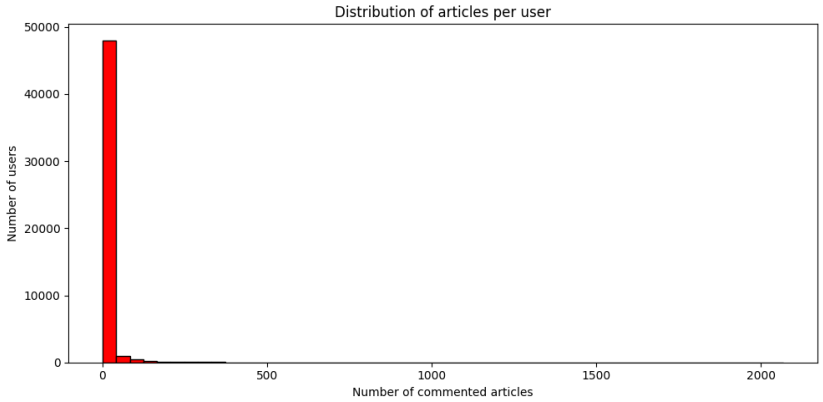


Figure 1: Distribution of articles per user.

edge weights without adding meaningful information.
When subsampling is enable, a random sample of SUBSAMPLE_USERS users is drawn from the total number of users. The sampling is random, meaning that every user has an equal probability of being selected regardless of how many articles they commented on. A fixed random seed ("seed == 42") is used to ensure that the same subset of users is selected across different executions, making the results fully reproducible.
The value of SUBSAMPLE_USERS acts as a global parameter: increasing it retains more users and produces a larger and denser graph at the cost of higher memory usage and longer execution times.
The filtered comments table is then merged with the articles table via a left join on "articleID" to retrieve the section column. This information is not relevant for PageRank algorithm, but is essential for the Topic-Sensitive PageRank, which requires knowing which section each article belongs to in order to define specific teleportation vectors. A final check removes any remaining rows with missing values in "userID" or "articleID".
An exploratory analysis was carried out to understand the structure of the data. The distribution of comments per article shows how many comments each article received, giving an indication of how much discussion it generates among users. The second distribution, the distribution of article per users, shows how many articles each user commented on, reflecting how each user partecipates in the platform. Both distribution are right-skewed: most articles receive few comments (median 11, max 712) and most users commented on very fer articles (median 1, max 2069), while a small number of articles and users dominate the dataset. The trend of these two distribution is shown in the figure 1 and 2. These statistic refer to an execution with SUBSAMPLE_USERS = 50000 and different values will generate different distribution.

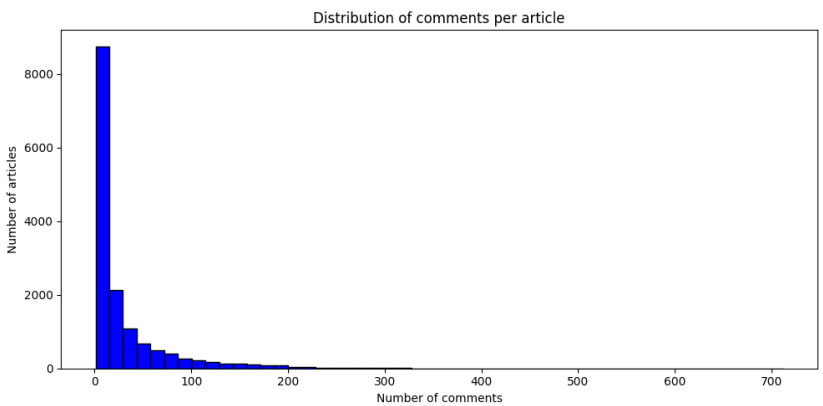


Figure 2: Distribution of comments per article.

# 5 Algorithms and implementations

## 5.1 PageRank

The PageRank algorithm was implemented from scratch using NumPy and SciPy sparse matrix operations and the implementations consists of two functions:

- build_transition_matrix(): it compute the column-normalized transition matrix M from the adjacency matrix A. Each column is divided by its sum, so that each column represents a probability distribution of transitioning from a node to its neighbors. Dead end nodes, that are nodes with no outgoing edges and identified by a zero column sum, are handled by temporarily setting their sum to 1 and leaving their column as all zero in M. This avoids division by zero while allowing their contribution to be redistributed at each iteration;

- pagerank(): it initializes the rank vector uniformly as $r_i = 1/N$ and iterates the following update rule:

$$r^{(t+1)} = \beta \cdot M \cdot r^{(t)} + \beta \cdot \frac{\sum_{i \in D} r_i^{(t)}}{N} \cdot \mathbf{1} + \frac{1-\beta}{N} \cdot \mathbf{1}$$

  where $\beta = 0.85$ is the probability that random surfer follows an edge at each iteration, D is set of dead end nodes and N the number of nodes. These three terms represent respectively: the rank propagated through the graph by following edges, the rank accumulated by dead end nodes redistributed uniformly to all nodes and the teleportation component that allows the random surfer to jump to any node with probability $1 - \beta$.
  Convergence is checked at each iteration by computing the L1 norm of the difference between consecutive rank vectors it is reached when this value falls below $\epsilon = 10^{-6}$ or after a maximum of 100 iterations.

## 5.2 PageRank with Spark

The Spark implementation follows the same mathematical formulation as the NumPy version but distributes the computation across multiple partitions using PySpark Resiliant Distributed Datasets (RDDs).
The adjacency matrix A is converted to Coordinate format (COO), which stores a sparse matrix as three array (row, cols and data) where each triple represents a non-zero entry of the matrix. This allow to easily extract the list of edges (src, dst) pairs, which are then parallelized into a Spark RDD. This is reorganized into an adjacency list RDD of the form (node, [neighbors]) using groupByKey(), deduplicated with distinct and cached in memory. The rank vector is represented as an RDD of (node, rank), initialized uniformly as $r_i = 1/N$.
The values of N, $\beta$ and the set of dead end nodes are distributed to all workers as broadcast variables, avoiding repeated data transfers across the network at each iteration.
Since Spark recomputes RDDs from scratch every time they are accessed, the adjacency list and rank RDDs are memorized with "MEMORY_AND_DISK", that means they are stored in memory or in disk if memory is insufficient. At each iteration, once the new rank RDD has been computed, the old one is unpersisted to free memory and avoid accumulation over many iterations.
At each iteration, the dead end node contribution and the teleportation term are computed as scalar values in the driver and combined into a single "base_rank". The rank propagation is then carried out in three steps: first, the adjacency list RDD is joined with the rank RDD; then each node distributes its rank equally among its neighbor; last, the received contributions are aggregated with combineKey() and scaled by $\beta$, adding "base_rank" to each result. A "leftOuterJoin()" with "all_nodes" ensures that nodes receiving no contributions are assigned "base_rank" rather being dropped.
Convergence is checked in the same way as it was done for PageRank.

## 5.3 Topic-Sensitive PageRank

The Topic-Sensitive PageRank extends the standard PageRank by replacing the uniform teleportation vector with a topic-specific one.
In this project, three topics are considered: "Opinion", "U.S." and "World". For each topic k, the set $T_k$ of users who commented on at least one article belonging to that section is identified. The teleportation vector is defined as:

$$\text{teleport}_i = \begin{cases} \frac{1}{|T_k|} & \text{if } i \in T_k \\ 0 & \text{otherwise} \end{cases}$$

The update rule becomes:

$$r^{(t+1)} = \beta \cdot M \cdot r^{(t)} + \beta \cdot (\sum_{i \in D} r_i^{(t)}) \cdot teleport + (1 - \beta) \cdot teleport$$

Compared to standard PageRank, both the teleportation component and the dead end nodes contribution are redistributed exclusively towards users active

in that specific topic.
The algorithm is run independently for each of three topics, using the same convergences criteria as the standard PageRank.

# 6 Scalability

The implemented solution is designed to scale to the full dataset through several concrete measures.
During data loading, the comments file is read in chunks of 100000 rows so that memory usage remains bounded regardless of file size and only the strictly necessary columns are loaded.The global variable "SUBSAMPLE" controls whether to work on a reduced subset or the full dataset: setting it to "False" is sufficient to process all data without any other code change.
During graph construction, the adjacency matrix is stored in CSR sparse format, whose memory usage grows with the number of edges. Edges are accumulated in buffers of 2000000 entries and added to matrix avoiding the need to hold all edges in memory at once. The cap on the number of users per article controls memory and computation by preventing the number of edges from growing quadratically for highly commented articles.

# 7 Experiments

Two experiments were conducted:

- PageRank comparison: it compares the NumPy and Spark PageRank implementations. Both were run on the same graph with $\beta = 0.85, \epsilon = 10^{-6}$ and a maximum of 100 iterations. The goals were to measure execution time, number of iterations to convergence and consistency of the top 20 ranking;

- Topic Sensitive and PageRank comparison: it runs independently the Topic-Sensitive PageRank for three topics ("Opinion", "U.S." and "World") and the goal is to compare the rankings against the normal implementation of PageRank to evaluate how the ranking changes based on the selected topic.

# 8 Results

The results of the two experiments are reported below:

- PageRank comparison: both implementation converged after 37 iterations producing identical top 20 rankings and scores. These results correspond to what was expected.
  The PageRank implemented using NumPy takes 6.63s to converge against 1497,59s (about 25 minutes) for Spark. This difference is expected because

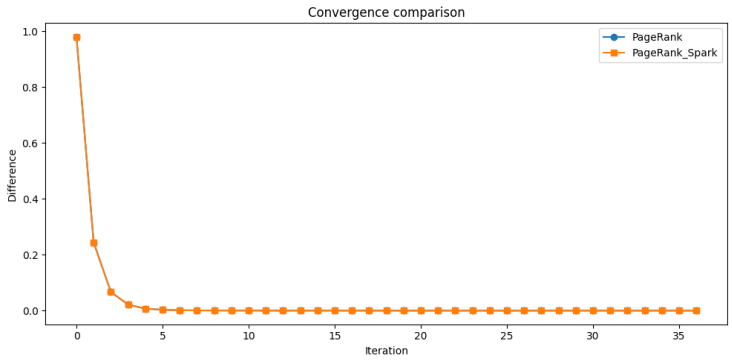


Figure 3: Comparison on convergence curve

| Rank | UserID | Score | PR Rank | TSPR_Opinion | TSPR_U.S. | TSPR_World |
|---|---|---|---|---|---|---|
| 1 | 61986282 | 0.001118 | 1 | 1 | 1 | 1 |
| 2 | 2073520 | 0.000912 | 2 | 2 | 2 | 2 |
| 3 | 51905269 | 0.000901 | 3 | 3 | 3 | 3 |
| 4 | 75716722 | 0.000779 | 4 | 7 | 5 | 4 |
| 5 | 33668404 | 0.000762 | 5 | 6 | 4 | 5 |
| 6 | 80110445 | 0.000748 | 6 | 5 | 6 | 6 |
| 7 | 75215278 | 0.000713 | 7 | 8 | 9 | 11 |
| 8 | 51415176 | 0.000682 | 8 | 9 | 8 | 9 |
| 9 | 37617930 | 0.000672 | 9 | 4 | 11 | 10 |
| 10 | 62938502 | 0.000658 | 10 | 10 | 10 | 12 |
| 11 | 80370332 | 0.000653 | 11 | 13 | 12 | 15 |
| 12 | 48376006 | 0.000650 | 12 | 14 | 7 | 7 |
| 13 | 51087612 | 0.000636 | 13 | 12 | 17 | 16 |
| 14 | 23255351 | 0.000623 | 14 | 39 | 25 | 21 |
| 15 | 57353517 | 0.000612 | 15 | 11 | 18 | 17 |
| 16 | 87637841 | 0.000607 | 16 | 47 | 45 | 27 |
| 17 | 73444633 | 0.000595 | 17 | 20 | 15 | 8 |
| 18 | 69548808 | 0.000594 | 18 | 29 | 13 | 14 |
| 19 | 83218918 | 0.000590 | 19 | 25 | 16 | 29 |
| 20 | 60927884 | 0.000572 | 20 | 19 | 21 | 13 |

Figure 4: Comparison on rankings of Topic-Sensitive and PageRank

Spark is designed for distributed systems but running it on a single machine makes it slower than a simple implementation of PageRank. The comparison graph of the convergence curves is shown in 3 which shows no difference as both implementation converge with the same number of iterations.

- Topic Sensitive and PageRank comparison: all the three topic reach convergence after 63 iterations, more than PageRank as the topic-specific teleportation slows convergence.
  The top 2 users retain the first two position across all three topics suggesting they are globally dominant regardless of topic. As the figure 4 shows, based on the topic analyzed, there may be changes in position or even disappearance from the top 20.